

# Cryptography Lab - SHA256, GPG, and Hash Identification

---

**Lab environment:** Kali Linux

**Scope:** Authorized local lab only

**Objective:** Hash a file with SHA256, modify one byte/character, prove the hash changes, encrypt/decrypt a text file with GPG, run Hashcat example hashes, and identify common hash types.

## 1. Command Summary with Purpose

### 1.1 Create a clean lab directory

```
mkdir -p ~/cyber-labs/day-crypto-lab  
cd ~/cyber-labs/day-crypto-lab  
pwd
```

**Purpose:**

Creates a separate folder for this lab and moves into it. `pwd` confirms your current working directory so your files do not mix with other labs.

### 1.2 Create a sample file

```
echo "This is my first cryptography lab file." > message.txt  
cat message.txt  
ls -l message.txt
```

**Purpose:**

Creates a plaintext test file, displays its content, and checks file details such as size and permissions.

### 1.3 Generate the original SHA256 hash

```
sha256sum message.txt  
sha256sum message.txt > original_hash.txt  
cat original_hash.txt
```

**Purpose:**

Generates the first SHA256 hash of `message.txt` and saves it as the baseline hash. This baseline is used later to detect whether the file changed.

### 1.4 Modify the file by adding one character

```
printf "!" >> message.txt  
cat message.txt  
sha256sum message.txt  
sha256sum message.txt > modified_hash.txt
```

**Purpose:**

Adds only one extra character to the file and generates a new SHA256 hash. This demonstrates that even a tiny change produces a completely different hash.

### 1.5 Compare original and modified hashes

```
cat original_hash.txt
cat modified_hash.txt
diff original_hash.txt modified_hash.txt
```

#### Purpose:

Displays both hashes and compares them. The `diff` output proves that the original and modified hash values are different.

### 1.6 Create two separate files for clean comparison

```
echo "This is my first cryptography lab file." > original_message.txt
cp original_message.txt modified_message.txt
printf "!" >> modified_message.txt
sha256sum original_message.txt modified_message.txt
sha256sum original_message.txt modified_message.txt > sha256_comparison.txt
cat sha256_comparison.txt
```

#### Purpose:

Creates an original file and a modified copy. This is a cleaner way to show both hashes side by side. It is useful for screenshots and portfolio evidence.

### 1.7 Verify file integrity with a checksum file

```
sha256sum original_message.txt > original_message.sha256
sha256sum -c original_message.sha256
```

#### Purpose:

Creates a checksum file and verifies that the original file still matches its saved SHA256 hash. Expected result: `original\_message.txt: OK`.

### 1.8 Modify the file and verify again

```
echo "modified" >> original_message.txt
sha256sum -c original_message.sha256
```

#### Purpose:

Changes the file after the checksum was created. The verification should fail, proving that SHA256 can detect file tampering or accidental modification.

### 1.9 Check or install GPG

```
gpg --version
sudo apt update
sudo apt install gnupg -y
```

#### Purpose:

Checks whether GPG is installed. If it is missing, the install commands add it to Kali. GPG is used for encryption and decryption.

### 1.10 Create a secret text file

```
echo "This is a secret message for my cybersecurity lab." > secret.txt  
cat secret.txt
```

#### Purpose:

Creates a plaintext file that will be encrypted using GPG.

### 1.11 Encrypt the file using GPG symmetric encryption

```
gpg -c secret.txt  
ls -l  
file secret.txt.gpg
```

#### Purpose:

Encrypts `secret.txt` using a passphrase and creates `secret.txt.gpg`. The `file` command confirms that the output is GPG encrypted data.

### 1.12 Remove plaintext and decrypt the encrypted file

```
rm secret.txt  
ls -l  
gpg -o decrypted_secret.txt -d secret.txt.gpg  
cat decrypted_secret.txt
```

#### Purpose:

Deletes the original plaintext file, then decrypts the `.gpg` file back into readable text. This proves that encryption is reversible with the correct passphrase.

### 1.13 Optional: Generate a GPG public/private key pair

```
gpg --full-generate-key  
gpg --list-keys
```

#### Purpose:

Creates an asymmetric GPG key pair. The public key can be shared, while the private key must stay secret. This is useful for understanding public-key cryptography.

### 1.14 Optional: Encrypt using a GPG public key

```
echo "This file is encrypted using my public key." > public_secret.txt  
gpg -e -r "Irfan Rafiq" public_secret.txt  
ls -l public_secret.txt*  
gpg -o public_decrypted.txt -d public_secret.txt.gpg  
cat public_decrypted.txt
```

#### Purpose:

Encrypts a file using a recipient public key and decrypts it using the matching private key. This demonstrates asymmetric encryption.

### 1.15 Check or install Hashcat

```
hashcat --version
sudo apt update
sudo apt install hashcat -y
```

#### Purpose:

Checks whether Hashcat is installed. If missing, it installs Hashcat. In this lab, Hashcat is used only for viewing example hash formats and identifying hash types.

### 1.16 Run Hashcat example hashes

```
hashcat --example-hashes
hashcat --example-hashes > hashcat_example_hashes.txt
head -n 40 hashcat_example_hashes.txt
```

#### Purpose:

Displays and saves Hashcat's built-in examples of supported hash types. This helps identify hash formats safely without cracking real passwords.

### 1.17 Search specific hash examples

```
grep -i "md5" hashcat_example_hashes.txt | head
grep -i "sha256" hashcat_example_hashes.txt | head
grep -i "bcrypt" hashcat_example_hashes.txt | head
grep -i "ntlm" hashcat_example_hashes.txt | head
```

#### Purpose:

Searches the saved Hashcat examples for common hash families. This helps you recognize different hash patterns.

### 1.18 Generate sample hashes locally

```
echo -n "password123" | md5sum
echo -n "password123" | sha256sum
echo -n "password123" | sha512sum
```

#### Purpose:

Creates local sample hashes for learning. `-n` prevents adding a newline, making the hash output consistent.

### 1.19 Install and use hashid

```
sudo apt update
sudo apt install hashid -y
echo -n "password123" | md5sum | awk '{print $1}' > md5_hash.txt
hashid $(cat md5_hash.txt)
echo -n "password123" | sha256sum | awk '{print $1}' > sha256_hash.txt
hashid $(cat sha256_hash.txt)
```

#### Purpose:

Installs `hashid`, saves clean hash values without filenames, and asks `hashid` to guess possible hash types. Hash identification is based on patterns, so it may show multiple possibilities.

## 1.20 Lookup Hashcat mode numbers

```
hashcat --help | grep -i "md5"  
hashcat --help | grep -i "sha256"  
hashcat --help | grep -i "ntlm"
```

### Purpose:

Finds Hashcat mode numbers for specific hash types. Example: MD5 is mode `0`, SHA2-256 is commonly mode `1400`, and NTLM is mode `1000`.

## 2. Paste-Ready Lab Notes for GitHub / Report

### Cryptography Lab - SHA256, GPG, and Hash Identification

#### Objective

The objective of this lab was to understand file hashing, file integrity verification, encryption and decryption using GPG, and basic hash type identification using Hashcat examples.

#### Tools Used

- Kali Linux
- sha256sum
- GPG
- Hashcat
- hashid

#### Task 1: SHA256 File Hashing

A test file was created and hashed using SHA256.

```
echo "This is my first cryptography lab file." > message.txt  
sha256sum message.txt
```

The SHA256 hash was saved as a baseline.

```
sha256sum message.txt > original_hash.txt
```

#### Explanation

SHA256 generates a fixed-length cryptographic hash. This hash acts like a digital fingerprint of the file.

#### Task 2: One-Character File Modification

One small change was made to the file.

```
printf "!" >> message.txt
```

The file was hashed again.

```
sha256sum message.txt > modified_hash.txt
```

Both hashes were compared.

```
diff original_hash.txt modified_hash.txt
```

## Observation

The original and modified hashes were completely different even though only one small change was made.

## Explanation

This demonstrates the avalanche effect. In cryptographic hashing, even a tiny change in input creates a completely different hash output. This makes SHA256 useful for file integrity verification.

## Task 3: File Integrity Verification

A checksum file was created.

```
sha256sum original_message.txt > original_message.sha256
```

The file was verified.

```
sha256sum -c original_message.sha256
```

Expected initial result:

```
original_message.txt: OK
```

After modifying the file, verification failed.

```
echo "modified" >> original_message.txt
```

```
sha256sum -c original_message.sha256
```

Expected result:

```
original_message.txt: FAILED
```

## Explanation

This proves that SHA256 can detect unauthorized or accidental file modification.

## Task 4: GPG Symmetric Encryption

A secret text file was created.

```
echo "This is a secret message for my cybersecurity lab." > secret.txt
```

The file was encrypted using GPG symmetric encryption.

```
gpg -c secret.txt
```

This created:

```
secret.txt.gpg
```

The original plaintext file was removed.

```
rm secret.txt
```

The encrypted file was decrypted.

```
gpg -o decrypted_secret.txt -d secret.txt.gpg
```

The decrypted content was verified.

```
cat decrypted_secret.txt
```

## Explanation

GPG symmetric encryption uses a passphrase to encrypt and decrypt a file. Without the correct passphrase, the encrypted file cannot be read.

## Task 5: Hashcat Example Hashes

Hashcat supported hash examples were displayed.

```
hashcat --example-hashes
```

The output was saved to a file.

```
hashcat --example-hashes > hashcat_example_hashes.txt
```

Different hash types were searched.

```
grep -i "md5" hashcat_example_hashes.txt | head
grep -i "sha256" hashcat_example_hashes.txt | head
grep -i "bcrypt" hashcat_example_hashes.txt | head
grep -i "ntlm" hashcat_example_hashes.txt | head
```

## Explanation

Hashcat provides example hashes for many supported formats. These examples are useful for learning how different hash types look.

## Task 6: Hash Type Identification

Sample hashes were generated locally.

```
echo -n "password123" | md5sum
echo -n "password123" | sha256sum
echo -n "password123" | sha512sum
```

Hash types were identified based on length and pattern.

| Hash Type          | Pattern                                            |
|--------------------|----------------------------------------------------|
| MD5                | 32 hexadecimal characters                          |
| SHA1               | 40 hexadecimal characters                          |
| SHA256             | 64 hexadecimal characters                          |
| SHA512             | 128 hexadecimal characters                         |
| bcrypt             | Starts with `\$2a\$`, `\$2b\$`, or `\$2y\$`        |
| Linux SHA512 crypt | Starts with `\$6\$`                                |
| NTLM               | 32 hexadecimal characters, usually Windows context |

## Explanation

Hash type identification depends on the hash length, prefix, format, and context. Tools like `hashid` can help, but their output should be treated as a best guess.

## 3. Key Learning

Hashing is used to verify integrity. Encryption is used to protect confidentiality. A hash cannot be reversed, while encrypted data can be decrypted with the correct key or passphrase. SHA256 is useful for detecting file changes. GPG can protect files using encryption. Hashcat supports many hash formats, and hash type identification depends on hash pattern, length, and context.

## 4. Interview-Style Explanation

In this lab, I created a file and generated its SHA256 hash. After modifying only one character, the hash changed completely. This demonstrated the avalanche effect and showed why hashes are useful for file integrity checks.

I also used GPG to encrypt and decrypt a text file. The encrypted file was unreadable until decrypted with the correct passphrase. This showed the difference between hashing and encryption: hashing is one-way, while encryption is reversible with the correct key.

Finally, I used Hashcat example hashes and hash identification techniques to recognize common hash types such as MD5, SHA256, SHA512, NTLM, and bcrypt.

## 5. Portfolio Output Format

```
cybersecurity-portfolio/  
└─ foundation/  
    └─ cryptography-lab/  
        ├── README.md  
        ├── sha256_comparison.txt  
        ├── original_hash.txt  
        ├── modified_hash.txt  
        ├── hashcat_example_hashes.txt  
        └─ screenshots/  
            ├── sha256-before-after.png  
            ├── sha256-integrity-ok-failed.png  
            ├── gpg-encrypt-decrypt.png  
            └─ hashcat-example-hashes.png
```

Recommended screenshots:

1. sha256sum before and after modification
2. sha256sum -c showing OK and FAILED
3. GPG encryption/decryption result
4. hashcat --example-hashes output
5. hashid identifying MD5/SHA256

## 6. Tracker Update Suggestion

Lab completed: Yes

Report completed: Partial

Portfolio completed: Crypto lab notes + screenshots added

Hours planned: 4

Hours done: 4

Status: Completed

Blocker / note: SHA256 integrity, GPG encryption/decryption, and hash type identification completed in Kali